0. Student Email and Identifier Validation

Problem Statement

Check student email-like strings and registration IDs using simple formatting rules. Report missing parts or invalid characters.

Objectives

• Check whether an email has one @ symbol and a dot after it.

• Check whether an ID contains only letters and digits, with no spaces.

Assumptions

• Email must contain one @, text before @, and a dot in the part after @.

• Registration ID must not be empty and may contain only letters and digits.

Algorithm

1. Read email and registration ID.

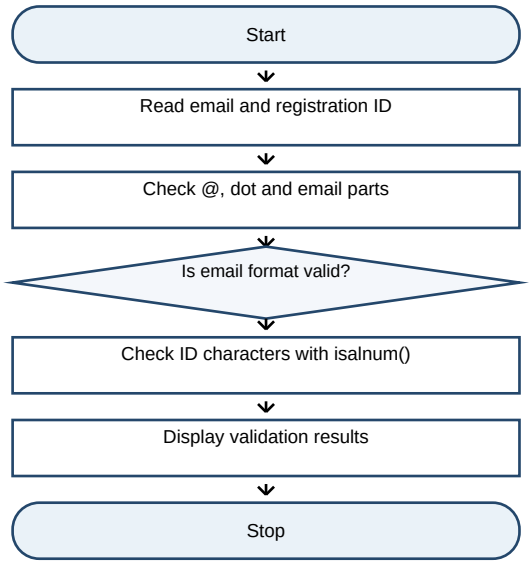
2. Count @ symbols and check the parts around @ and dot.

3. Check each ID character using isalnum().

4. Print valid or invalid messages.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT email, reg_id
Check email format
Check every character in reg_id
DISPLAY results
STOP
```

Python Source Code

```
email = input("Enter email: ")
reg_id = input("Enter registration ID: ")
email_ok = email.count("@") == 1
if email_ok:
    parts = email.split("@")
    email_ok = parts[0] != "" and "." in parts[1] and parts[1] != ""
id_ok = reg_id != ""
```

```
for ch in reg_id:
    if not ch.isalnum():
        id_ok = False
if email_ok:
    print("Email format is valid")
else:
    print("Email format is invalid")
if id_ok:
    print("Registration ID is valid")
else:
    print("Registration ID is invalid")
```

Test Cases

Input	Expected check	Expected result
student@college.edu	STU123	Both valid
studentcollege.edu	STU 123	Both invalid

Sample Output

```
Enter email: student@college.edu
Enter registration ID: STU123
Email format is valid
Registration ID is valid
```

Conclusion

Basic string checks can identify common formatting mistakes in email-like strings and registration IDs.

1. String-Based Examination Answer Evaluation

Problem Statement

Compare each student's answers with an answer key, count correct, incorrect and unanswered responses, and calculate a score with negative marking.

Objectives

• Compare answers position by position.

• Apply marks for correct answers and a penalty for wrong answers.

Assumptions

• Answers are entered as letters separated by spaces.

• Correct answer = +1 mark, wrong answer = -0.25 mark, blank answer = 0.

Algorithm

1. Read the answer key and student answers.

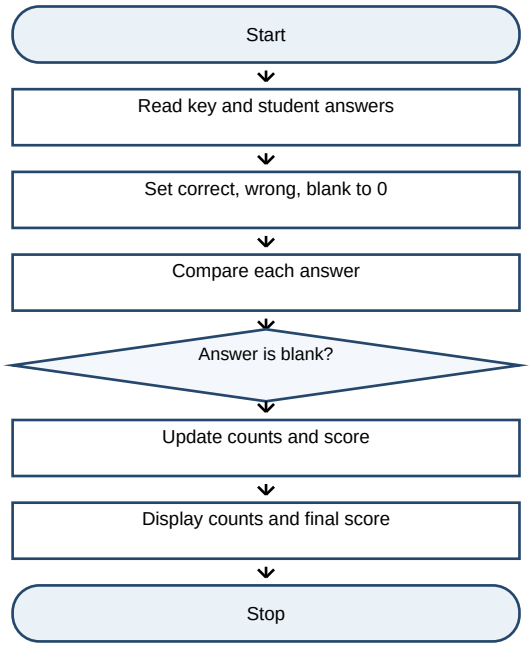
2. Compare each answer with the matching key answer.

3. Count correct, wrong and blank answers.

4. Calculate score and display the result.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT key and answers
FOR each answer
Count correct, wrong or blank
score = correct - wrong * 0.25
DISPLAY result
STOP
```

Python Source Code

```
key = input("Enter answer key (use spaces): ").upper().split()
answers = input("Enter student answers (use spaces; use - for blank): ").upper().split()
correct = 0
```

```
wrong = 0
blank = 0
for i in range(len(key)):
    if i >= len(answers) or answers[i] == "-":
        blank = blank + 1
    elif answers[i] == key[i]:
        correct = correct + 1
    else:
        wrong = wrong + 1
score = correct - wrong * 0.25
print("Correct:", correct)
print("Incorrect:", wrong)
print("Unanswered:", blank)
print("Score:", round(score, 2))
```

Test Cases

Input	Expected check	Expected result
A B C D / A X - D	2 correct, 1 wrong, 1 blank	Score = 1.75

Sample Output

```
Enter answer key (use spaces): A B C D
Enter student answers (use spaces; use - for blank): A X - D
Correct: 2
Incorrect: 1
Unanswered: 1
Score: 1.75
```

Conclusion

Comparing strings with a loop helps calculate examination results in a simple and consistent way.

2. Text-Based Invoice Data Extraction

Problem Statement

Read invoice records written in a fixed format and extract product code, quantity and amount. Report records that do not follow the format.

Objectives

• Separate fields using a delimiter.

• Check that quantity and amount are numeric.

Assumptions

• Each record uses the format CODE,QUANTITY,AMOUNT.

• Fields are separated by commas and amount can be a whole number or decimal.

Algorithm

1. Read one invoice record.

2. Split the record using commas.

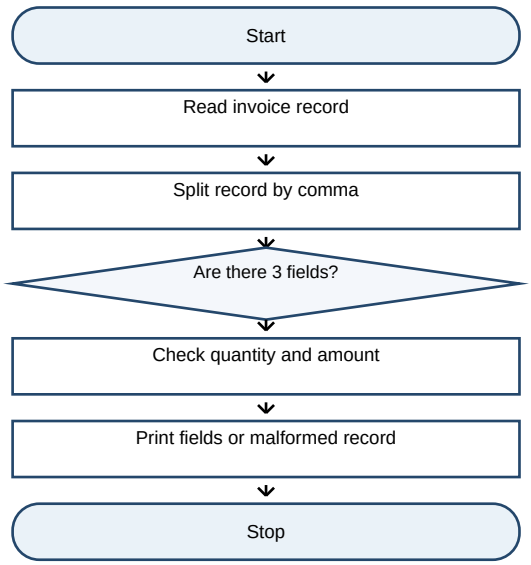
3. Check that there are three fields.

4. Check quantity and amount are numeric.

5. Display extracted values or an error.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT record
fields = split record by comma
IF field count is 3 and numbers are valid
DISPLAY extracted values
ELSE DISPLAY malformed record
STOP
```

Python Source Code

```
record = input("Enter record (CODE,QUANTITY,AMOUNT): ")
fields = record.split(",")
if len(fields) == 3:
    code = fields[0].strip()
    quantity = fields[1].strip()
```

```
    amount = fields[2].strip()
    if code != "" and quantity.isdigit():
        try:
            amount_value = float(amount)
            if amount_value >= 0:
                print("Product code:", code)
                print("Quantity:", int(quantity))
                print("Amount:", amount_value)
            else:
                print("Malformed record")
        except ValueError:
            print("Malformed record")
    else:
        print("Malformed record")
else:
    print("Malformed record")
```

Test Cases

Input	Expected check	Expected result
PEN12,3,45.50	3 fields; numbers valid	Product code PEN12, quantity 3, amount 45.5
PEN12,three,45	Quantity is not numeric	Malformed record

Sample Output

```
Enter record (CODE,QUANTITY,AMOUNT): PEN12,3,45.50
Product code: PEN12
Quantity: 3
Amount: 45.5
```

Conclusion

Simple splitting and numeric checks can extract useful information from invoice text.

3. Campus Registration Identifier Analyzer

Problem Statement

Check student registration identifiers, extract branch and admission-year codes, and classify records.

Objectives

• Check identifier length and structure.

• Read branch and year codes from fixed positions.

Assumptions

• Identifier format is YYBRNNN, for example 24CS123.

• First two characters are year, next two are branch, last three are digits.

Algorithm

1. Read the identifier and convert it to uppercase.

2. Check length is 7.

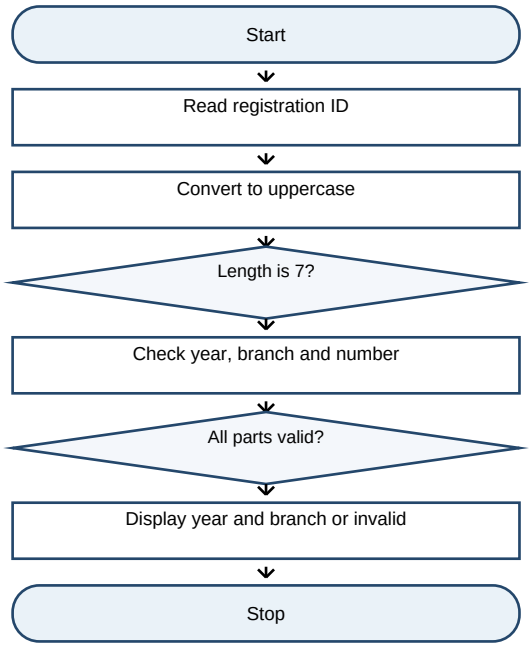
3. Check year and final digits are numeric.

4. Check branch contains letters.

5. Display the year and branch if valid.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT reg_id
Check length and character groups
IF valid, extract year and branch
DISPLAY classification
ELSE DISPLAY invalid
STOP
```

Python Source Code

```
reg_id = input("Enter registration ID (example 24CS123): ").upper()
if len(reg_id) == 7:
    year = reg_id[0:2]
```

```
    branch = reg_id[2:4]
    number = reg_id[4:7]
    if year.isdigit() and branch.isalpha() and number.isdigit():
        print("Valid ID")
        print("Admission year: 20" + year)
        print("Branch code:", branch)
    else:
        print("Invalid ID")
else:
    print("Invalid ID")
```

Test Cases

Input	Expected check	Expected result
24CS123	7 characters and correct groups	Valid; year 2024; branch CS
24C9123	Branch contains a digit	Invalid ID

Sample Output

```
Enter registration ID (example 24CS123): 24CS123
Valid ID
Admission year: 2024
Branch code: CS
```

Conclusion

Fixed-position string slicing makes it easy to examine a campus registration ID.

4. Student Feedback Sentiment Scoring

Problem Statement

Analyse feedback sentences using given positive and negative words, calculate a score, and classify the feedback.

Objectives

• Count positive and negative keywords.

• Classify feedback using the score.

Assumptions

• Words are separated by spaces and punctuation is removed for this simple example.

• Score = positive count minus negative count. Score above 0 is Positive, below 0 is Negative, otherwise Neutral.

Algorithm

1. Read feedback and convert it to lowercase.

2. Replace common punctuation with spaces.

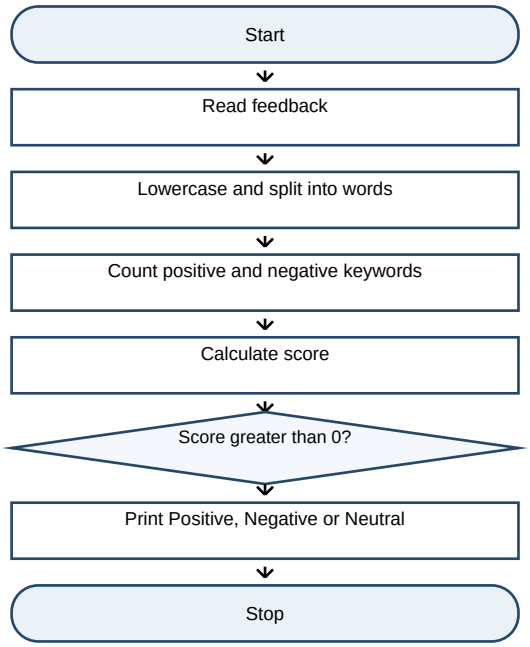
3. Split feedback into words.

4. Count positive and negative keywords.

5. Calculate score and print classification.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT feedback
Convert to lowercase and make word list
Count positive and negative words
score = positive - negative
Classify score and display
STOP
```

Python Source Code

```
feedback = input("Enter feedback: ").lower()
feedback = feedback.replace(".", " ").replace(",", " ").replace("!", " ")
words = feedback.split()
```

```
positive_words = ["good", "great", "helpful", "excellent"]
negative_words = ["bad", "poor", "slow", "boring"]
positive = 0
negative = 0
for word in words:
    if word in positive_words:
        positive = positive + 1
    if word in negative_words:
        negative = negative + 1
score = positive - negative
print("Positive words:", positive)
print("Negative words:", negative)
print("Score:", score)
if score > 0:
    print("Feedback: Positive")
elif score < 0:
    print("Feedback: Negative")
else:
    print("Feedback: Neutral")
```

Test Cases

Input	Expected check	Expected result
The class is good and helpful	2 positive, 0 negative	Score 2; Positive
The service is slow and poor	0 positive, 2 negative	Score -2; Negative

Sample Output

```
Enter feedback: The class is good and helpful
Positive words: 2
Negative words: 0
Score: 2
Feedback: Positive
```

Conclusion

A keyword list and simple counting can provide a basic sentiment score for student feedback.

5. Text Compression Using Character Runs

Problem Statement

Compress a string by replacing consecutive repeated characters with the character and its count, then compare lengths.

Objectives

• Find consecutive repeated characters.

• Compare the original and encoded string lengths.

Assumptions

• Each run is written as character followed by count, for example aaabb becomes a3b2.

• This simple method is for learning and is not suitable for every text.

Algorithm

1. Read the string.

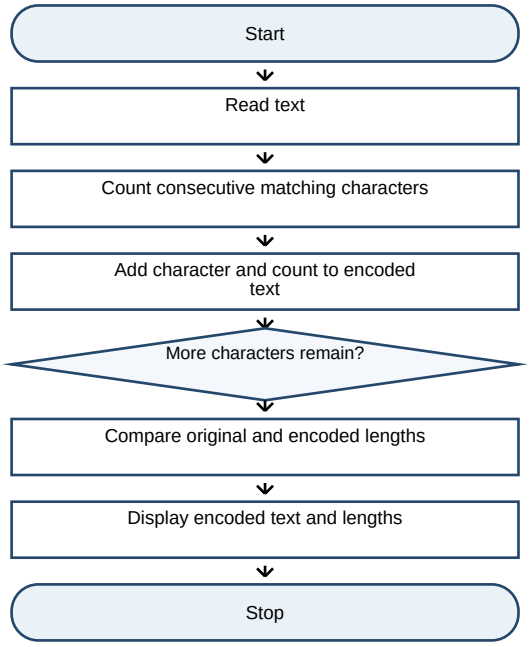
2. Start with the first character and count repeats.

3. Add each character and its count to the encoded string.

4. Compare both lengths and display the result.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT text
Build runs of same characters
Add character and count
Compare lengths
DISPLAY encoded text and lengths
STOP
```

Python Source Code

```
text = input("Enter text: ")
encoded = ""
i = 0
while i < len(text):
```

```
    count = 1
    while i + count < len(text) and text[i] == text[i + count]:
        count = count + 1
    encoded = encoded + text[i] + str(count)
    i = i + count
print("Original:", text)
print("Encoded:", encoded)
print("Original length:", len(text))
print("Encoded length:", len(encoded))
if len(encoded) < len(text):
    print("Encoding is shorter")
else:
    print("Encoding is not shorter")
```

Test Cases

Input	Expected check	Expected result
aaabbbbcc	a3b4c2	Original 9; encoded 6; shorter
abcd	a1b1c1d1	Original 4; encoded 8; not shorter

Sample Output

```
Enter text: aaabbbbcc
Original: aaabbbbcc
Encoded: a3b4c2
Original length: 9
Encoded length: 6
Encoding is shorter
```

Conclusion

Run-length encoding reduces length when many characters repeat consecutively, but can increase length otherwise.

6. Repeated Word and Phrase Detection

Problem Statement

Count words in a paragraph without considering uppercase and lowercase as different, and show repeated words with their positions.

Objectives

• Normalize letter case and count word occurrences.

• Record the positions of repeated words.

Assumptions

• Words are separated by spaces; commas and full stops are removed.

• Positions start from 1.

Algorithm

1. Read a paragraph and convert it to lowercase.

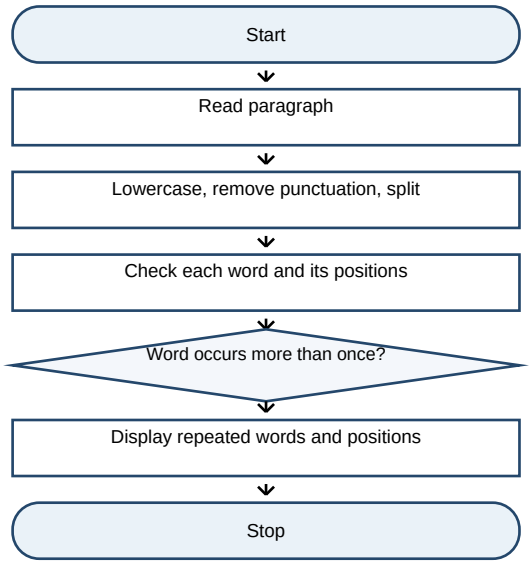
2. Remove common punctuation and split into words.

3. For each word, count its occurrences in the word list.

4. Display words that occur more than once and their positions.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT paragraph
Normalize text and create word list
FOR each word, find its positions
IF repeated, DISPLAY word and positions
STOP
```

Python Source Code

```
paragraph = input("Enter a paragraph: ").lower()
paragraph = paragraph.replace(",", "").replace(".", "").replace("!", "")
words = paragraph.split()
checked = []
for word in words:
    if word not in checked:
        positions = []
```

```
    for i in range(len(words)):
        if words[i] == word:
            positions.append(i + 1)
    if len(positions) > 1:
        print(word, "positions:", positions)
    checked.append(word)
```

Test Cases

Input	Expected check	Expected result
Good day, good work.	good at 1 and 3	Repeated word good at positions [1, 3]

Sample Output

```
Enter a paragraph: Good day, good work.
good positions: [1, 3]
```

Conclusion

Lists and loops can find repeated words and show where they appear in a paragraph.

7. Nested Loop Calendar Generator

Problem Statement

Print a monthly calendar using the starting weekday and number of days in the month.

Objectives

• Align dates under the correct weekday.

• Use loops to print the calendar.

Assumptions

• Weekday number is 0 for Sunday, 1 for Monday, through 6 for Saturday.

• Number of days must be between 28 and 31.

Algorithm

1. Read the number of days and starting weekday.

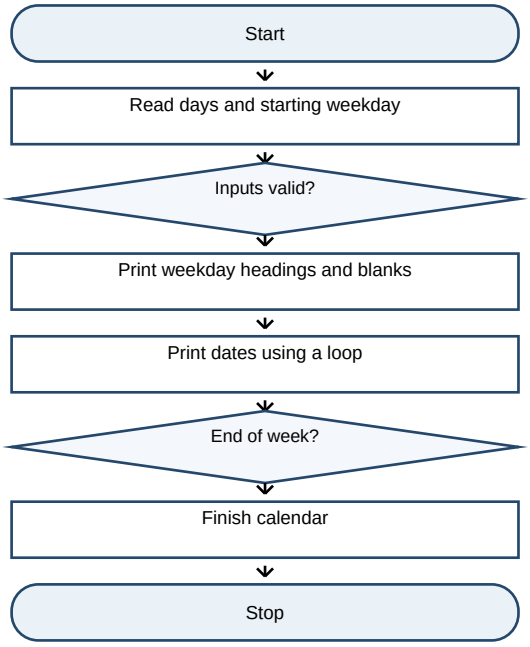
2. Print weekday headings.

3. Print blank spaces before day 1.

4. Print each day and move to a new row after Saturday.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT days, start_day
Check input ranges
Print weekday headings
Print blank cells before day 1
FOR day from 1 to days, print day
STOP
```

Python Source Code

```
days = int(input("Enter number of days (28 to 31): "))
start_day = int(input("Starting weekday (0=Sun, 6=Sat): "))
if 28 <= days <= 31 and 0 <= start_day <= 6:
    print("Su Mo Tu We Th Fr Sa")
```

```
    for i in range(start_day):
        print("   ", end="")
    for day in range(1, days + 1):
        print("%2d" % day, end=" ")
        if (start_day + day) % 7 == 0:
            print()
    print()
else:
    print("Invalid input")
```

Test Cases

Input	Expected check	Expected result
Days=30, start weekday=0	Starts on Sunday	Dates 1 to 30 printed in aligned rows
Days=35, start weekday=0	Days outside range	Invalid input

Sample Output

```
Enter number of days (28 to 31): 30
Starting weekday (0=Sun, 6=Sat): 0
Su Mo Tu We Th Fr Sa
1 2 3 4 5 6 7
8 9 10 11 12 13 14
15 16 17 18 19 20 21
22 23 24 25 26 27 28
29 30
```

Conclusion

Nested loops and weekday calculations help print a simple monthly calendar.

8. ATM Menu-Driven Banking Simulation

Problem Statement

Create a menu-based ATM program for deposits, withdrawals, balance checks and transaction summaries.

Objectives

• Repeat menu choices until the user exits.

• Validate amounts and prevent withdrawals above the balance.

Assumptions

• Starting balance is ■1000.

• Each deposit or withdrawal must be greater than zero.

• Maximum withdrawal per transaction is ■5000.

Algorithm

1. Set starting balance and transaction counters.

2. Display menu repeatedly.

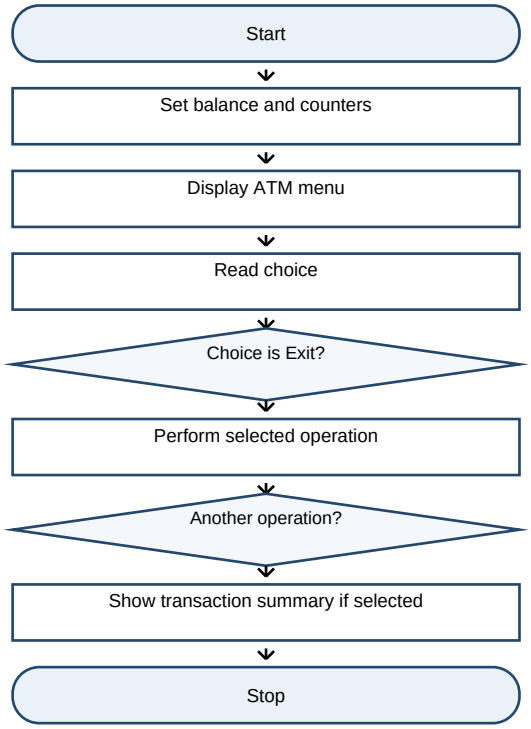
3. Perform the selected operation after validating it.

4. Display transaction summary when requested.

5. Stop when the user chooses Exit.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
balance = 1000
REPEAT show menu and read choice
Perform deposit, withdrawal, balance or summary
UNTIL choice is Exit
STOP
```

Python Source Code

```
balance = 1000
deposits = 0
withdrawals = 0
while True:
    print("1. Deposit")
    print("2. Withdraw")
    print("3. Check balance")
    print("4. Transaction summary")
    print("5. Exit")
    choice = input("Enter choice: ")
    if choice == "1":
        amount = float(input("Enter deposit amount: "))
        if amount > 0:
            balance = balance + amount
            deposits = deposits + 1
            print("Deposit successful")
        else:
            print("Invalid amount")
    elif choice == "2":
        amount = float(input("Enter withdrawal amount: "))
        if amount > 0 and amount <= 5000 and amount <= balance:
            balance = balance - amount
            withdrawals = withdrawals + 1
            print("Withdrawal successful")
        else:
            print("Withdrawal not allowed")
    elif choice == "3":
        print("Balance:", balance)
    elif choice == "4":
        print("Deposits:", deposits)
        print("Withdrawals:", withdrawals)
        print("Balance:", balance)
    elif choice == "5":
        print("Thank you")
        break
    else:
        print("Invalid choice")
```

Test Cases

Input	Expected check	Expected result
Deposit 500 from 1000	Valid deposit	Balance becomes 1500
Withdraw 2000 from 1000	Amount exceeds balance	Withdrawal not allowed

Sample Output

```
1. Deposit
2. Withdraw
3. Check balance
4. Transaction summary
5. Exit
Enter choice: 3
Balance: 1000
```

Conclusion

A while loop and if-elif statements can simulate basic ATM operations and enforce simple limits.

9. Multi-Stage Examination Eligibility System

Problem Statement

Decide whether a student can attend the final examination based on attendance, lab completion, internal marks and fee clearance.

Objectives

• Check all eligibility conditions.

• Report every requirement that is not satisfied.

Assumptions

• Attendance must be at least 75%.

• All laboratory work must be completed (yes/no).

• Internal marks must be at least 18 out of 30.

• Fees must be cleared (yes/no).

Algorithm

1. Read attendance, lab status, internal marks and fee status.

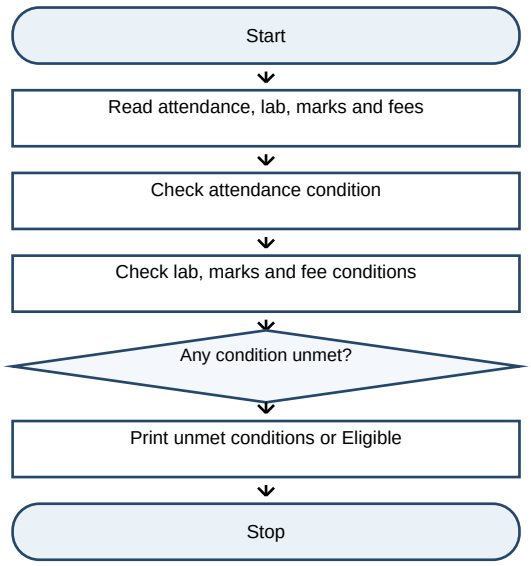
2. Check each condition separately.

3. Print every unmet condition.

4. If none are unmet, declare the student eligible.

Flowchart

Oval = Start/Stop; rectangle = process; diamond = decision.



Pseudocode

```
START
INPUT all eligibility details
Create list of unmet requirements
Check attendance, lab, marks and fees
IF list is empty, DISPLAY Eligible
ELSE DISPLAY each unmet requirement
STOP
```

Python Source Code

```
attendance = float(input("Enter attendance percentage: "))
lab = input("Are all labs completed? (yes/no): ").lower()
marks = int(input("Enter internal marks out of 30: "))
```

```
fees = input("Are fees cleared? (yes/no): ").lower()
unmet = []
if attendance < 75:
    unmet.append("Attendance must be at least 75%")
if lab != "yes":
    unmet.append("Complete all laboratory work")
if marks < 18:
    unmet.append("Internal marks must be at least 18")
if fees != "yes":
    unmet.append("Clear the fees")
if len(unmet) == 0:
    print("Eligible for final examination")
else:
    print("Not eligible")
    for item in unmet:
        print("-", item)
```

Test Cases

Input	Expected check	Expected result
Attendance 80, labs yes, marks 22, fees yes	All conditions satisfied	Eligible for final examination
Attendance 65, labs no, marks 15, fees no	Four conditions fail	Not eligible; all four reasons displayed

Sample Output

```
Enter attendance percentage: 65
Are all labs completed? (yes/no): no
Enter internal marks out of 30: 15
Are fees cleared? (yes/no): no
Not eligible
- Attendance must be at least 75%
- Complete all laboratory work
- Internal marks must be at least 18
- Clear the fees
```

Conclusion

Checking each rule separately helps explain clearly why a student is or is not eligible.